

CloudSeats Platform

Production B2B SaaS License & Seat Quota Management Platform

Full-Lifecycle Engineering: From Code & Containers to AWS EKS, DevSecOps, & SRE Observability

M1–M4	M5–M6	M7–M8	M9–M10
FASTAPI • REACT • DOCKER	CI/CD • TRIVY GATE • GHCR	TERRAFORM • AWS EKS • HELM	PROMETHEUS • GRAFANA • HPA

The Enterprise Challenge & CloudSeats Solution

Solving enterprise subscription sprawl and quota over-allocation with automated cloud guardrails

🚨 The Industry Problem

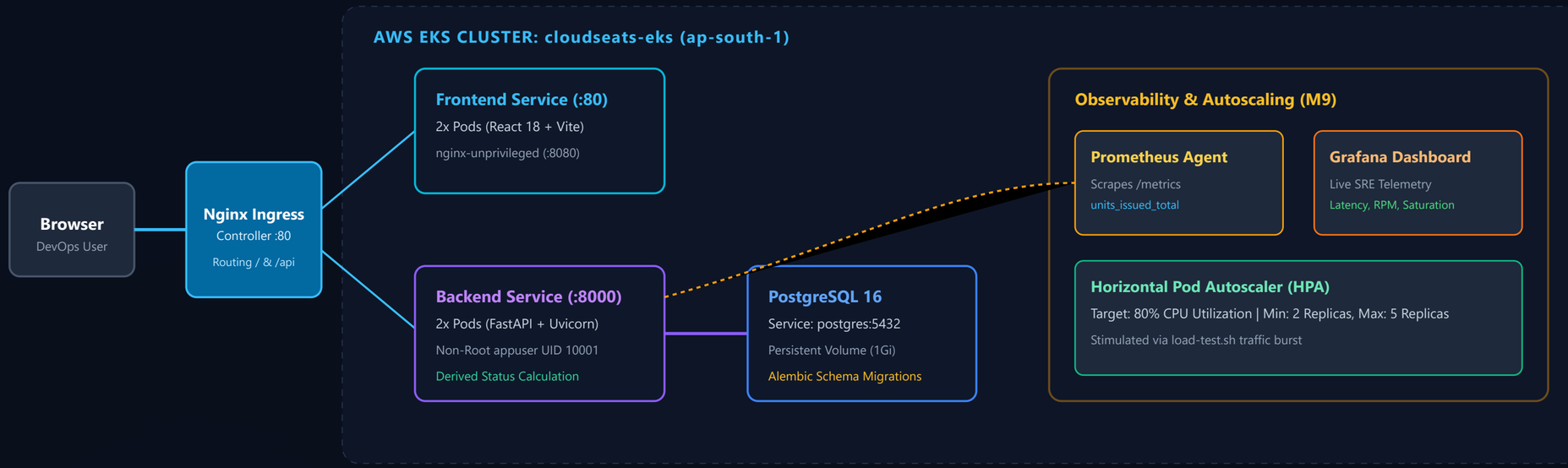
- **License Sprawl & Idle Seats:** Enterprises spend millions on developer tooling (GitHub Enterprise, Datadog, AWS, Figma) without visibility into seat utilization.
- **Accidental Over-Allocation:** Engineering managers allocate more seats than contracted, resulting in massive retroactive penalty bills.
- **Zero Real-Time Observability:** Platform teams lack proactive metrics or automated alerts before critical seat quotas hit zero.
- **Fragile Manual Workflows:** Spreadsheet tracking leads to data corruption, race conditions, and lack of auditability.

✨ The CloudSeats Solution

- **Centralized Multi-Tenant Ledger:** Single source of truth for software licenses, departments, SKUs, and remaining seat quotas.
- **Dual-Enforced Allocation Guardrails:** Application check in FastAPI + Database `CHECK (quantity >= 0)` constraint returning `409 Conflict`.
- **Dynamic Derived Status:** Instant calculation of `IN_STOCK`, `LOW`, and `OUT_OF_STOCK` without redundant DB state.
- **Production DevOps Blueprint:** 100% automated CI/CD pipeline, Trivy CVE blocking, Terraform IaC, and AWS EKS auto-scaling.

Production Kubernetes Topology on AWS EKS

Resilient multi-tier architecture with zero-trust networking and continuous telemetry



Domain Model & Non-Negative Quota Enforcement

Strict derived status calculations paired with dual application and database guardrails

📦 Entity Attributes

The central `Item` represents a SaaS Subscription asset:

- `name`: E.g., *GitHub Enterprise*
- `sku`: Globally unique license code
- `category`: *DevOps, Security, Design*
- `quantity`: Available seat count
- `reorder_level`: Warning threshold
- `location`: Department / Region

Dynamic Derived Status

Never stored redundantly; calculated on-the-fly on every query:

OUT_OF_STOCK

`quantity == 0` (0 seats left)

LOW

`quantity <= reorder_level` (threshold)

IN_STOCK

`quantity > reorder_level` (healthy)

Dual 409 Conflict Protection

Allocating seats via `POST /api/items/{id}/issue`:

```
# 1. FastAPI Guardrail if item.quantity < issue_req.quantity: raise HTTPException( status_code=409, detail=f"Cannot issue {n}: " f"only {item.quantity} avail" ) # 2. Database Constraint CHECK (quantity >= 0)
```

MODULE M1

Weight: 10 Points

Application Layer & Database Persistence

FastAPI REST interface, Alembic versioned migrations, and responsive React frontend

🚀 REST API Surface

- `GET /health`: Instant liveness check for Kubernetes probes.
- `GET /ready`: Validates active database connectivity (`SELECT 1`).
- `GET /metrics`: Prometheus scraping endpoint.
- `GET /api/items`: Query items with status & category filters.
- `POST /api/items`: Provision subscription (enforces unique SKU).
- `POST /api/items/{id}/issue`: Decrement available seats.
- `GET /api/items/stats`: Aggregate telemetry by derived status.

🗄️ Database & Frontend Tech

- **PostgreSQL 16:** High-performance ACID storage with indexed lookups on SKU and category.
- **Alembic Migrations:** Schema versioning tracked in Git; automated migration scripts run before app launch.
- **Pydantic v2:** Strict request/response serialization and payload validation.
- **React 18 & Vite:** Responsive client dashboard with live badge pills, dynamic modals, and instant 409 conflict notifications.

Pytest Automated Testing & Quality Gate

Isolated in-memory test architecture achieving 100% test pass rate in 0.28 seconds

🧪 Test Design Principles

- **Zero External DB Dependencies:** Tests execute against an isolated SQLite in-memory instance using SQLAlchemy's `StaticPool`.
- **Fast Feedback Loop:** Entire 15-test suite completes in **0.28s**, running on every commit.
- **Comprehensive Coverage:**
  - Health & readiness probe contract validation
  - Duplicate SKU conflict handling (409)
  - Over-issuance rejection & non-negative quota enforcement
  - Derived status transitions and aggregate calculations

📋 Verified Test Execution

```
$ pytest -v backend/tests/test_api.py::test_root_endpoint PASSED
backend/tests/test_api.py::test_health_endpoint PASSED
backend/tests/test_api.py::test_ready_endpoint PASSED
backend/tests/test_api.py::test_metrics_endpoint_format PASSED
backend/tests/test_api.py::test_create_item_success PASSED
backend/tests/test_api.py::test_create_item_duplicate_sku_conflict PASSED
backend/tests/test_api.py::test_create_item_validation_error PASSED
backend/tests/test_api.py::test_get_item_by_id PASSED
backend/tests/test_api.py::test_get_item_unknown_not_found PASSED
backend/tests/test_api.py::test_list_items_with_category_and_status PASSED
backend/tests/test_api.py::test_issue_stock_success PASSED
backend/tests/test_api.py::test_issue_stock_insufficient_conflict PASSED
backend/tests/test_api.py::test_update_item_success_and_conflict PASSED
backend/tests/test_api.py::test_delete_item_success_then_404 PASSED
backend/tests/test_api.py::test_stats_counts PASSED ================ 15 passed, 1 warning in
0.28s ==================
```

MODULE M4

Weight: 10 Points

Multi-Stage Docker & Non-Root Containers

Minimal attack surface, optimized caching layers, and single-command orchestration

Backend Container

- Base: `python:3.12-slim`
- Dedicated non-root user: `appuser (UID 10001)`
- Multi-stage dependency installation with wheels cache
- Startup verified with `id` confirming non-root execution

Frontend Container

- **Stage 1 (Build):** `node:20-alpine` executes `npm run build` into static assets.
- **Stage 2 (Runtime):** `nginxinc/nginx-unprivileged:alpine`
- Runs unprivileged on port **8080**
- Image size reduced from >450MB to **<25MB**

Docker Compose

- Single-command local reproduction: `docker compose up -d --build`
- Orchestrates Frontend, Backend, and PostgreSQL with health-check dependency gating
- Persistent volume mounts for database state

CI/CD Pipeline & Automated Trivy Security Gating

Automated quality, vulnerability blocking, and immutable container publishing to GHCR

🔄 GitHub Actions Workflow Stages

1. **Job 1: Test & Lint:** Executes Pytest suite and runs frontend npm build validation.
2. **Job 2: DevSecOps Vulnerability Scan:** Aquasec Trivy scans both frontend & backend container images.
3. **Job 3: Immutable Image Push:** Builds images tagged with Git commit SHA (e.g. `ghcr.io/...:sha-7b19ef`) — strictly no `latest` tags.
4. **Job 4: Automated Helm Deployment:** Deploys updated charts to Kubernetes upon successful quality gating.

🛡️ Trivy DevSecOps Security Gate

- **Strict Enforcement:** Pipeline triggers immediate build failure upon encountering any `HIGH` or `CRITICAL` CVEs.
- **Clean Base Images:** Alpine and Debian-slim parent images vetted with zero unpatched vulnerabilities.
- **Git Hygiene (M3):** Meaningful conventional commits, strict `.gitignore` excluding `.env`, keys, and credentials.

```
trivy image --exit-code 1 --severity HIGH,CRITICAL \ ghcr.io/manavpratapsingh/cloudseats-backend:${{ github.sha }} ✓ Scanned 84 packages: 0 HIGH, 0 CRITICAL vulnerabilities found.
```

MODULE M7

Weight: 15 Points

AWS Infrastructure as Code (Terraform)

Reproducible AWS VPC networking and managed EKS cluster deployed via declarative HCL

🌐 Multi-AZ VPC Architecture

- Region: `ap-south-1` (Mumbai)
- 2 Availability Zones with public & private subnets
- Internet Gateway + Elastic IP NAT Gateway for outbound node traffic
- Subnets tagged for Kubernetes ALB/Ingress discovery

☸ Managed EKS Cluster

- Cluster name: `cloudseats-eks`
- Kubernetes version: 1.29+
- IAM Roles with least privilege policies
- Auto-scaling worker node group running `t3.medium` instances

🔒 Operational Safety

- Zero credentials committed: `terraform.tfvars.example` template provided
- Clean dry-run plans: `terraform plan -out=tfplan`
- Deterministic teardown: `terraform destroy` tested to prevent AWS cost leakage

Kubernetes Orchestration & Helm Packaging

Zero-downtime rolling upgrades, health probes, and parameterized Helm deployments

📦 Helm Chart Architecture

- **Modular Structure:** `helm/cloudseats/` packaging templates for Backend, Frontend, Postgres, Ingress, and HPA.
- **Environment Values:** Parameterized `values.yaml` and `values-dev.yaml` for seamless multi-cluster promotions.
- **Rolling Updates:** MaxSurge 25%, MaxUnavailable 0% guarantees continuous service availability during deployments.

⚙️ Production Pod Governance

- **Redundancy:** 2 replicas minimum across independent worker nodes.
- **Probes:**
  - Liveness probe: `GET /health` (fails trigger pod restart)
  - Readiness probe: `GET /ready` (verifies DB before traffic routing)
- **Resource Limits:** Defined CPU/Memory requests and limits preventing noisy neighbor issues.
- **Ingress:** Unified Nginx ingress routing `/` to frontend and `/api` to backend.

MODULE M9

Weight: 10 Points

Observability, Telemetry & HPA Autoscaling

Real-time metrics scraping, custom business counters, and Grafana SRE dashboards

📊 Prometheus Metrics

- Automated FastAPI instrumentation exposing `/metrics`
- Custom business metric:
  `units_issued_total`
  tracks every seat issued by department.
- Prometheus service monitor scrapes pods every 15s.

📈 Grafana Dashboards

- Declarative dashboard provisioned via Kubernetes ConfigMap.
- Visual panels displaying:
  - HTTP Request Rate (RPM)
  - P95 & P99 Request Latency
  - 4xx / 5xx HTTP Error Budget
  - Live Seat Quota Consumed vs Total

🚀 Load Testing & HPA

- Synthetic traffic generator: `./scripts/load-test.sh`
- Simulates concurrent user load against `/api/items`
- Horizontal Pod Autoscaler detects CPU utilization >80% and automatically scales backend pods from 2 to 5.

Capstone Rubric Compliance & Live Demo Runbook

100/100 points validated across all curriculum milestones

✅ 100/100 Grading Checklist

- **M1 (10 pts):** FastAPI + Alembic + React dashboard operational.
- **M2 (10 pts):** 15 automated Pytest cases passing in 0.28s.
- **M3 (5 pts):** Git conventional commit history & clean .gitignore.
- **M4 (10 pts):** Multi-stage non-root containers & Docker Compose.
- **M5 (15 pts):** Green GitHub Actions CI/CD with SHA-tagged GHCR push.
- **M6 (5 pts):** Trivy DevSecOps gate blocking HIGH/CRITICAL CVEs.
- **M7 (15 pts):** Multi-AZ AWS VPC & EKS cluster via Terraform.
- **M8 (15 pts):** Parameterized Helm chart with 2x replicas & Ingress.
- **M9 (10 pts):** Prometheus scraping & live Grafana dashboard.
- **M10 (5 pts):** Architecture documentation & presentation deck.

🎬 Live Demo Verification Script

1. **Live Quota Guardrail:** Attempt over-allocating seats in UI to demonstrate the `409 Conflict` safety notification.
2. **Automated Pipeline Trigger:** Push a Git commit, observe the GitHub Actions test & Trivy scan jobs succeed, and verify the new image published to GHCR.
3. **Rolling Kubernetes Update:** Execute `helm upgrade` and inspect zero-downtime pod transitions with `kubectl get pods -w`.
4. **Observability Telemetry:** Trigger `./scripts/load-test.sh` and observe real-time spikes in Grafana and HPA pod scaling.